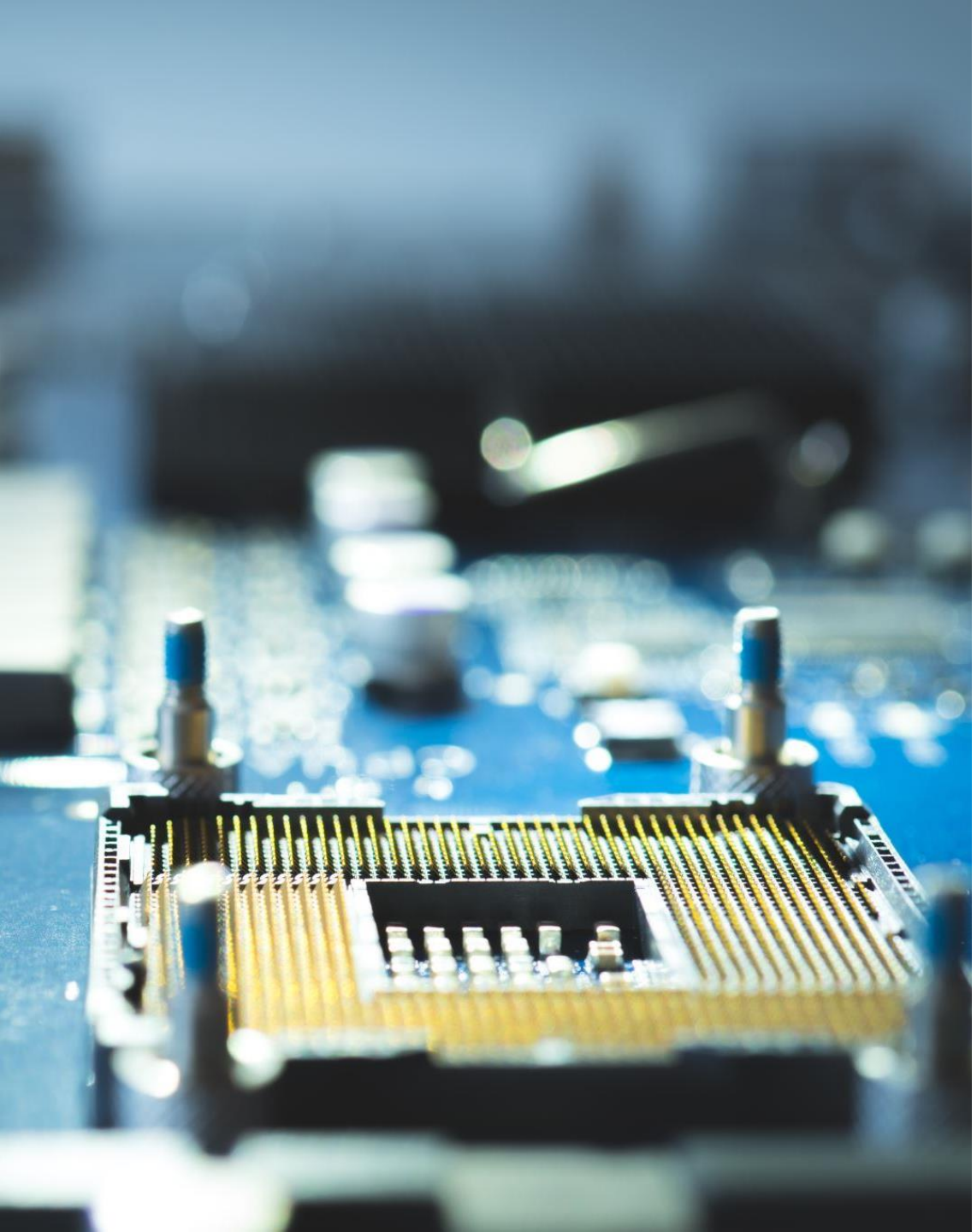
Infrastructure as Code CDK Practices

Ron C. Chiang

University of St. Thomas



CDK LAB:
Website on EC2 instance

- Create a new project directory

```
mkdir cdk-lab-web-server
cd cdk-lab-web-server
```

- Initialize a new app and stack

```
cdk init app --language python
source .venv/bin/activate
python -m pip install -r requirements.txt
```

VPC

- In cdk_lab_web_server_stack.py

```
# Create a VPC. CDK by default creates and attaches internet gateway for VPC
cdk_lab_vpc = ec2.Vpc(
  self,
  "cdk_lab_vpc",
  ip_addresses=ec2.IpAddresses.cidr("10.0.0.0/16"),

subnet_configuration=[ec2.SubnetConfiguration(name="PublicSubnet01",subnet_type=ec2.Subnet
Type.PUBLIC)]
)
```

Instance IAM role

```
# Instance Role and SSM Managed Policy
InstanceRole = iam.Role(self, "InstanceSSM",
  assumed_by=iam.ServicePrincipal("ec2.amazonaws.com"))


InstanceRole.add_managed_policy(iam.ManagedPolicy.from_aws_managed_policy_name("Amaz
  onSSMManagedInstanceCore"))
```

EC2 instance

```
# Create an EC2 instance
cdk_lab_web_instance = ec2.Instance(self,
"cdk_lab_web_instance",
vpc=cdk_lab_vpc,
instance_type=ec2.InstanceType("t2.micro"),
machine_image=ec2.AmazonLinuxImage(generation=ec2.AmazonLinuxGeneration.AMAZON_LINUX_
  2),
role=InstanceRole)
```

Userdata

- We need to install and configure a webserver because our EC2 instance uses a plain Linux OS.
- EC2 userdata can download scripts from S3 buckets.
- Steps:
  - Create a script called configure.sh to install a LAMP server.
  - Save configure.sh in your stack folder
  - Use Asset construct to upload the configure.sh to a bucket
  - Set the path to the script and tell userdata to use this script
  - Allow the instance to access the asset bucket

Security Groups

- EC2 instance's `connections` attribute is for controlling traffics in and out the instance, which is basically the firewall or security group

- To open port 80 for HTTP traffic

```
cdk_lab_web_instance.connections.allow_from_any_ipv4(ec2.Port.tcp(80))
```

- To open port 443 for HTTPS traffic

```
cdk_lab_web_instance.connections.allow_from_any_ipv4(ec2.Port.tcp(443))
```

Deploy

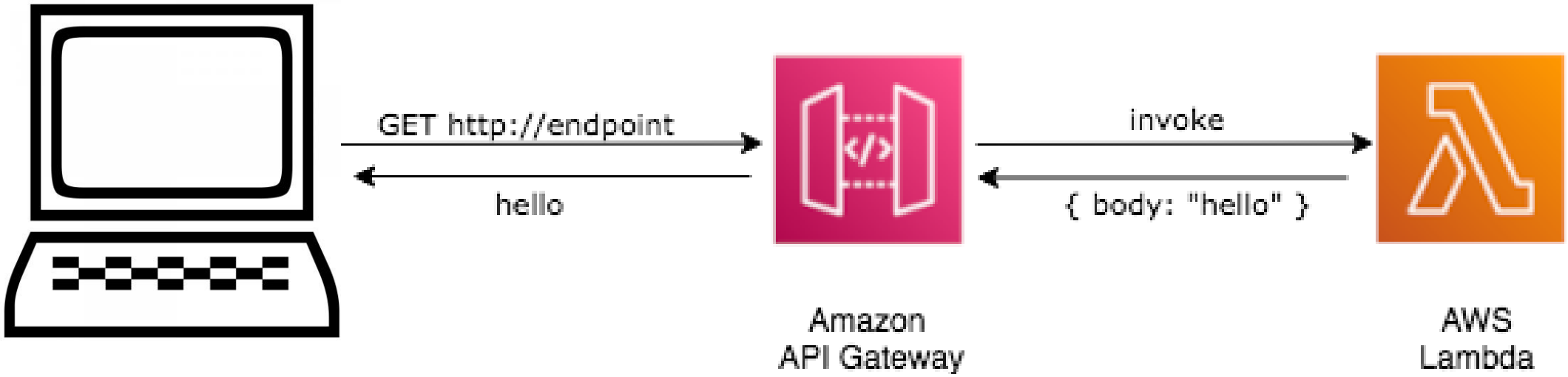
- Synthesize the app
  - cdk synth
- Deploy the app
  - cdk deploy
- Examine the resources
  - EC2 instance, VPC, subnet, security group, bucket
  - Test the website

CDK LAB: Serverless Web App

- Create a new project directory
  - `mkdir cdk-lab-serverless-web`
  - `cd cdk-lab-serverless-web`

- Initialize a new app and stack
  - `cdk init app --language python`
  - `source .venv/bin/activate`
  - `python -m pip install -r requirements.txt`

LAB Overview



Lambda handler code

- Create a directory called `lambda` in the root of the project tree .
- Add a file called `hello.py` (in the `lambda` directory):

```
import json

def handler(event, context):
    print('request: {}'.format(json.dumps(event)))
    return {
        'statusCode': 200,
        'headers': {
            'Content-Type': 'text/plain'
        },
        'body': 'Hello, CDK! You have hit {}\n'.format(event['path'])
    }
```

Add an AWS Lambda Function to the stack

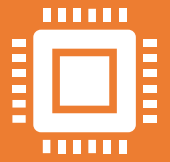
- First, we need to import `aws_lambda` library
- Then, use the Lambda construct to define functions

```
# Defines an AWS Lambda resource
        my_lambda = _lambda.Function(
            self, 'HelloHandler',
            runtime=_lambda.Runtime.PYTHON_3_12,
            code=_lambda.Code.from_asset('lambda'),
            handler='hello.handler',
        )
```

Test the function

- Before we move on to next component, let's make sure the function works
- First, deploy the stack. In this lab, you'll notice that `cdk deploy` not only deployed your stack, but also archived and uploaded the lambda directory from your disk to the bootstrap bucket
- Test the function
  - Open the AWS Lambda Console (make sure you are in the correct region)
  - Click on the **Test** button to open the Configure test event dialog
  - Select **Amazon API Gateway AWS Proxy** from the Event template list
  - Create and test the event

API Gateway



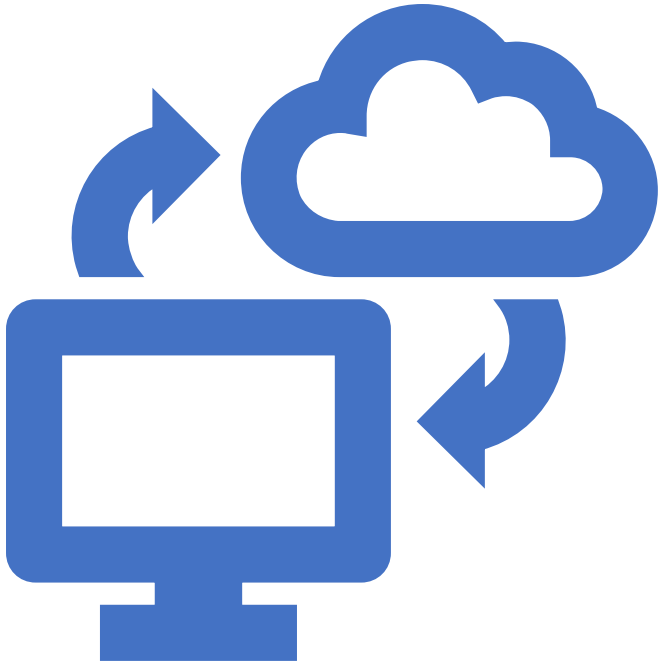
API Gateway will expose a public HTTP endpoint that anyone on the internet can visit with an HTTP client



We will use Lambda proxy integration



Any request to any URL path will be proxied directly to our Lambda function, and the response from the function will be returned back to the user



LambdaRestApi construct

- Import `aws_apigateway apigw`
- Define an API endpoint and associate it with our Lambda function

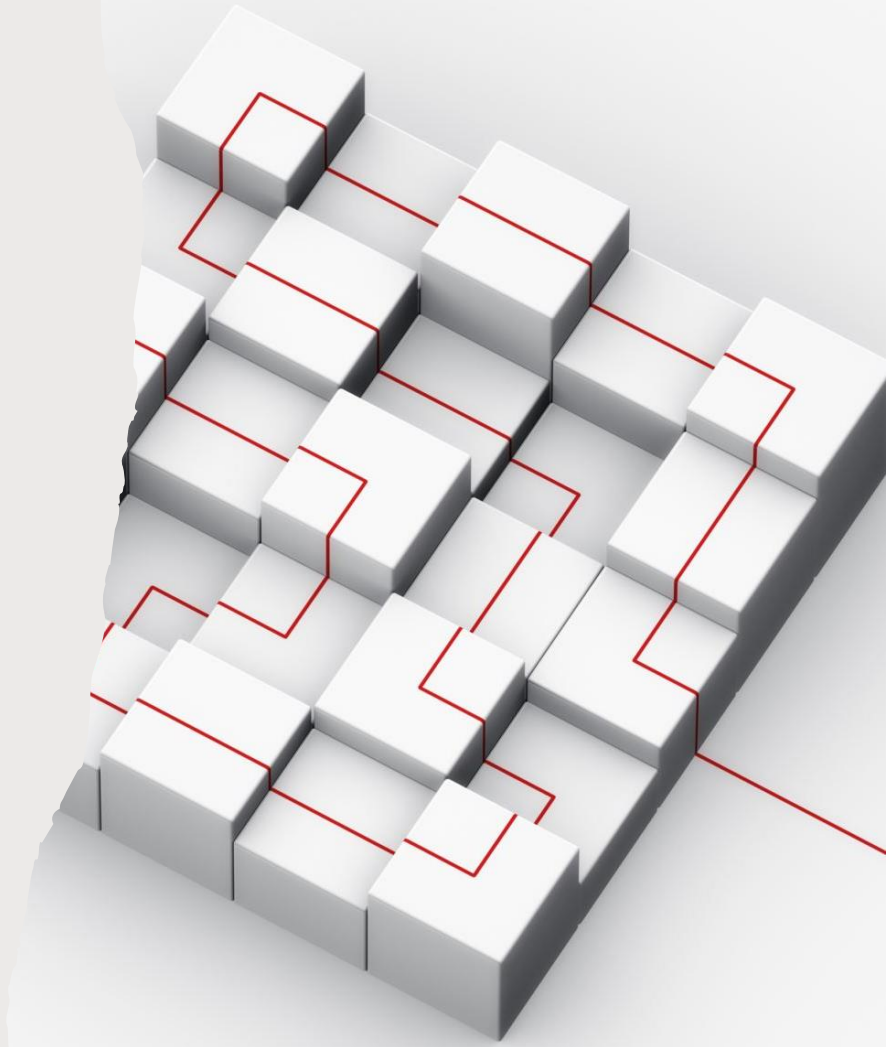
```
apigw.LambdaRestApi(
    self, 'Endpoint',
    handler=my_lambda
)
```

Testing the app

- Let's use `cdk diff` to see the changes we are going to make

- The output of `cdk deploy` will include the endpoint to the API Gateway
  - This is a stack output that's automatically added by the API Gateway construct and includes the URL of the API Gateway endpoint.

- Test the endpoint use any HTTP client (e.g., curl or any web browser)



Not working?

- To troubleshoot:
  - Make sure your handler returns a response that includes
    - statusCode,
    - body
    - header fields
  - Check CloudWatch logs for this Lambda function



Writing constructs

- In addition to using existing constructs, you can also write your own constructs, and then anyone can use them in their apps.

- All constructs are equal in the AWS CDK. An AWS CDK construct (such as s3.Bucket or sns.Topic) behaves the same as a construct from a third-party library published via NPM, Maven, or PyPI.

- To declare a new construct, create a class that extends the Construct base class, in the constructs package, then follow the pattern for initializer arguments.





CDK LAB:

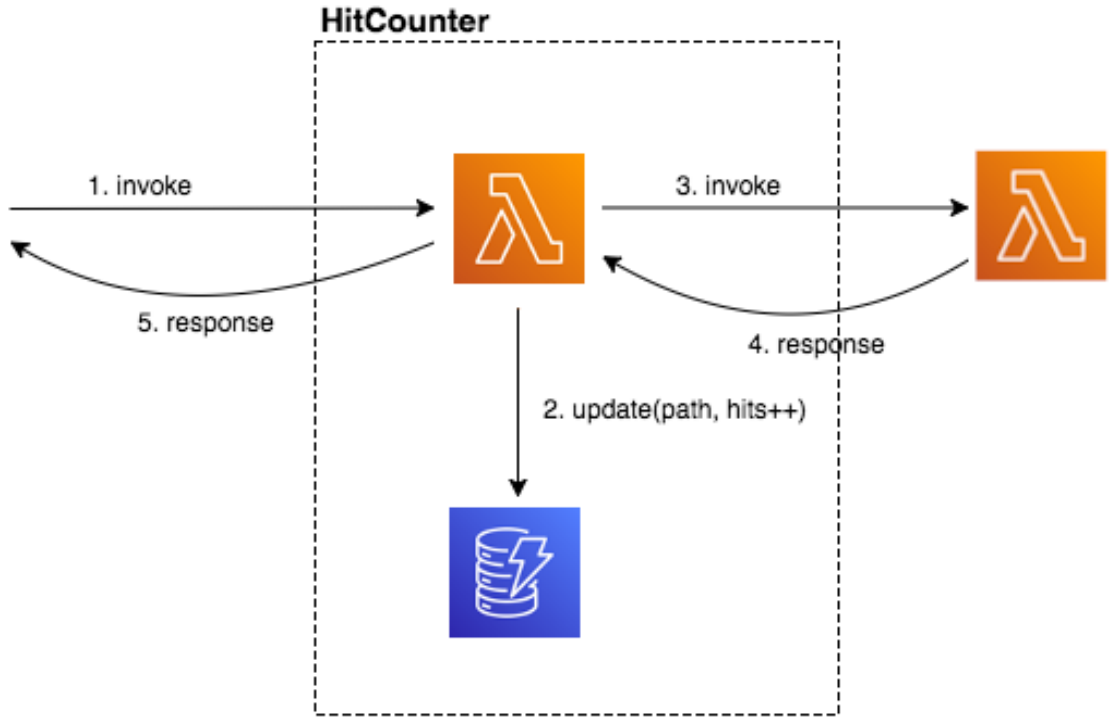
Writing Constructs

Extending the Serverless Web App with a Hit Counter

HitCounter construct

- Define a new construct called `HitCounter` and attach it to the API Gateway's backend Lambda function.

- The `HitCounter` construct will count how many requests were issued to each URL path and store it in a DynamoDB table.



Basic Structure of a Construct

- In `cdk-lab-serverless-web/cdk_lab_serverless_web`
- Create a file call `hitcounter.py` with
  - A new construct class called `HitCounter`
  - As usual, constructor arguments are *scope*, *id* and *kwargs*, and we propagate them to the cdk.Construct base class
  - The `HitCounter` class also takes one explicit keyword parameter `downstream` of type `lambda.IFunction`
    - This is where we are going to “plug in” the Lambda function that we just created so it can be hit-counted
- Before we finish this construct class, let's work on the handler code of this hit counter

Hit Counter Lambda Handler Setting

- This handler function uses a Dynamo DB table to track the number of visits to the site. (See the architecture diagram on a previous page)

- It also responses to the site visit through a lambda function

- Therefore, in our `lambda` directory, add a file called `hitcount.py` with the following items:
  - Import json, os, and boto3
  - Create a Dynamo DB table and a lambda client

AWS CDK vs AWS SDK



Cloud Development Kit (CDK)

AWS CDK is a framework that lets developers define and provision cloud infrastructure using code. It's an infrastructure-as-code tool that allows developers to define AWS resources in code. For example, developers can use CDK to write code to create AWS resources like API Gateway, DynamoDB, or Lambda



Software Development kit (SDK)

AWS SDK is a set of tools for developers to interact with AWS services. For example, fetching items from DynamoDB or listing the contents of an S3 bucket

Hit Counter Lambda Handler Statements

- In the Hit Counter Lambda Handler
  - Increase the hit count and update the Dynamo DB table
  - Invoke our previous Lambda function to show the web content
- To implement these tasks, this Lambda Handler relies on environment variables to get
  - The Dynamo DB table name
  - The name of Lambda function for showing the response
- Since the names of the Dynamo DB table and the Lambda function will be known at runtime, we need to go back to our HitCounter construct to wire up these values

Add resources to the hit counter construct

- Go back to hitcounter.py. Then,
  - Define a DynamoDB table with path as the partition key (every DynamoDB table must have a single partition key)
  - Define a Lambda function which is bound to the lambda/hitcount.handler code
  - wired the Lambda’s environment variables to the names of the table and the response function

Late-bound values

- The names of the table and response function are known when the stack is deployed
- If you print their values during synthesis, you will get a “TOKEN”, which is how the CDK represents these late-bound values
- You should treat tokens as *opaque strings*. This means you can concatenate them together for example, but don’t be tempted to parse them in your code

Add a hit counter to our stack

- Our HitCounter construct is ready. We need to add it to our stack
  - cdk-lab-serverless-web/cdk_lab_serverless_web/cdk_lab_serverless_web_stack.py
- Import the HitCounter construct
- Instantiate a hit counter with my_lambda as a downstream function
- Update API Gateway to use the Lambda handler with a counter

Deploy and Test

- cdk synth
- cdk deploy
- Test the endpoint in a web browser
- Is everything OK? If not, how to troubleshoot?

Granting Permissions

- Allow Lambda to access our DynamoDB table
  - In hitcounter.py, add the following statement:

    ```
    table.grant_read_write_data(self._handler)
    ```

- Deploy and Test again

- We also need to allow the HitCounter construct to invoke the downstream lambda function
  - `downstream.grant_invoke(self._handler)`

Test the hit counter

- Issue a few test requests
  - You can append any path at the end of the testing URL
- View the table in DynamoDB console

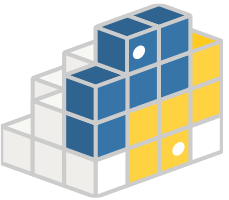




Using Construct Lib

- We just learned how to write our own construct.
- Now, let's see how to import a construct library to use it in this project.
- The goal of next task is to allow users to view DynamoDB tables in web browser

CDK DynamoDB Table Viewer



- The Python Package Index (PyPI) is a repository of software for the Python programming language.

- PyPI helps you find and install software developed and shared by the Python community.

- We will use a DynamoDB table viewer published on PyPI
  - https://pypi.org/project/cdk-dynamo-table-view/

Install the Construct Library

- Append requirements.txt with
  - `cdk-dynamo-table-view==0.2.488`

- Install the required dependencies
  - `pip install -r requirements.txt`

- The last two lines of the output should look like this:
  - `Installing collected packages: cdk-dynamo-table-view`
  - `Successfully installed cdk-dynamo-table-view-0.2.0`

Add the table viewer

- Add a `TableViewer` construct in `cdk_lab_serverless_web_stack.py`

```
from cdk_dynamo_table_view import TableViewer
```

- Instantiate a table viewer

```
TableViewer(
    self, 'ViewHitCounter',
    title='Hello Hits',
    table=??????
)
```

Add a table property to our hit counter

- Edit `hitcounter.py` and modify it as such table is exposed as a public property.

- Add a table property

```
@property
def table(self):
    return self._table
```

- Update all table related statements to use `self._table`

Access the table from our stack

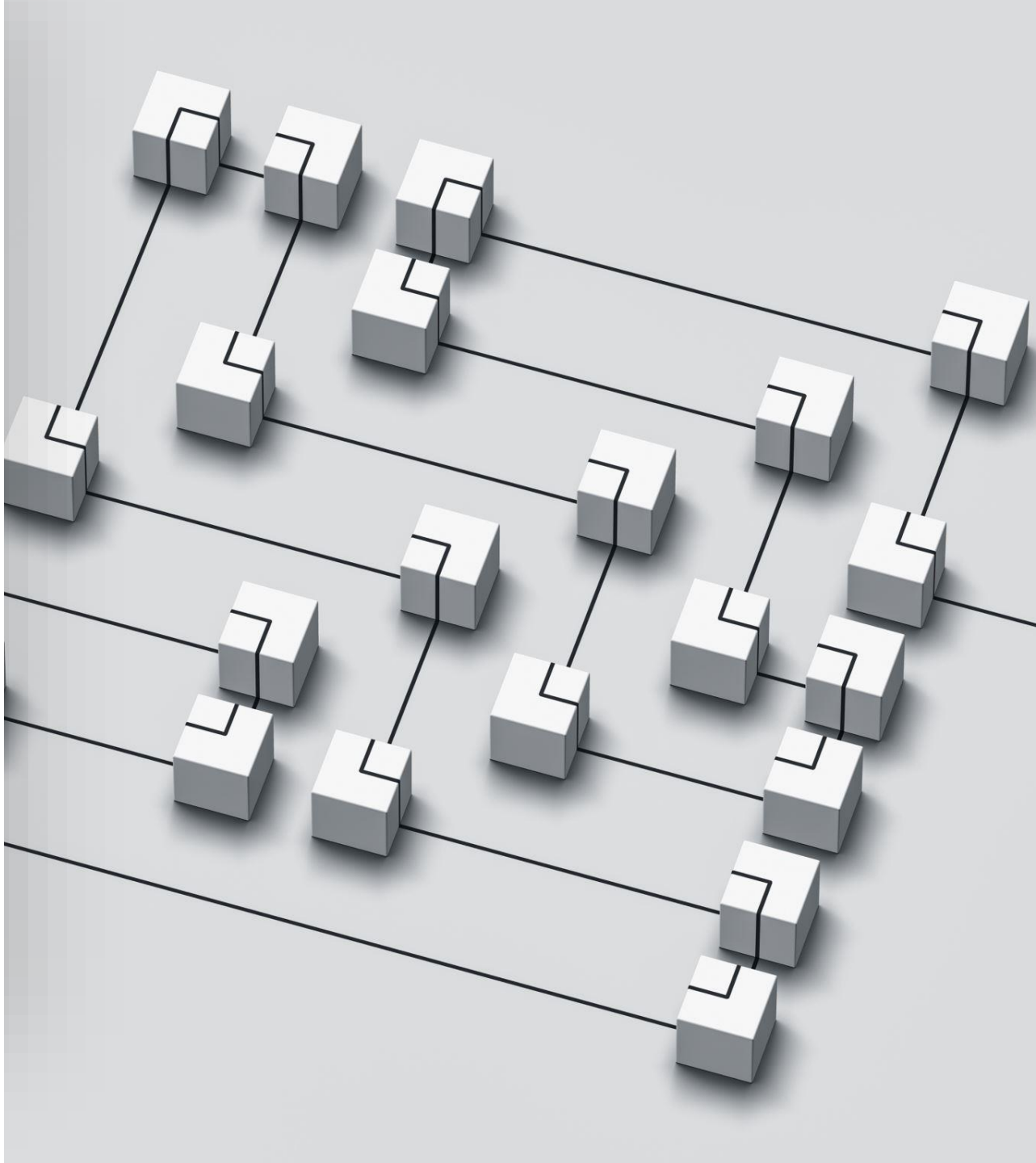
- Go back to `cdk_lab_serverless_web_stack.py`

- Assign the table property of the table viewer

```
TableViewer(
    self, 'ViewHitCounter',
    title='Hello Hits',
    table=hello_with_counter.table,
)
```

Deploy and Test

- cdk diff
  - See the difference
- cdk deploy. You'll see one more output for the viewer endpoint
- Test the endpoint by sending requests with various paths





Construct Lib

- Construct libraries allow you to add complex capabilities to your apps with minimum effort
- However, you must understand that with great power comes great responsibility.
- It is your responsibility to understand how certain constructs that you use impact your application
- Make sure you only use construct libraries from vendors you trust

Clean Up

- By default, most resources will get deleted upon stack deletion, however that's not the case for all resources

- The DynamoDB table will be retained by default. If you don't want to retain this table, we can set this in CDK code by using `RemovalPolicy`



Add RemovalPolicy

- Goal: Set the DynamoDB table to be deleted upon stack deletion
- In hitcounter.py
  - `import aws_cdk.RemovalPolicy`
  - Set _table's `removal_policy` to `RemovalPolicy.DESTROY`
- Deploy the updated stack.
- We can now enter `cdk destroy`



Is it clean?

- Lambda functions are gone. But their CloudWatch logs are permanently retained.
  - CloudWatch logs are not tracked by CloudFormation since they are not part of the stack, so they will persist.
  - You will have to manually delete these in the console if desired.
- What is that CDKToolkit CloudFormation stack?
  - Created by cdk bootstrap
  - Keep it until you really don't need to use CDK
  - If you want to remove CDKToolkit stack
    - Manually delete it in CloudFormation console
    - The S3 bucket created will be retained by default, so head to the S3 console and empty + delete the bucket generated from bootstrapping.

The philosophy behind the AWS CDK

- Failure patterns often arise during the deployment and ongoing maintenance of complex cloud applications.
- Failures are often related to "out-of-band" changes to an application that aren't fully tested, such as configuration changes.
- The AWS CDK provides a model to define your entire application in code, not only business logic but also infrastructure and configuration.
- That way, proposed changes can be carefully reviewed, comprehensively tested in environments resembling production to varying degrees, and fully rolled back if something goes wrong.

Coding best practices

- Start simple and add complexity only when you need it
- Align with the AWS Well-Architected Framework
- Every application starts with a single package in a single repository
  - don't put multiple applications in the same repository
- Move code into repositories based on code lifecycle or team ownership
- Infrastructure and runtime code live in the same package

Construct best practices

- Model with constructs, deploy with stacks
- Configure with properties and methods, not environment variables
- Unit test your infrastructure
- Don't change the logical ID of stateful resources
- Constructs aren't enough for compliance

Application best practices

- Make decisions at synthesis time
- Use generated resource names, not physical names
- Define removal policies and log retention
- Separate your application into multiple stacks as dictated by deployment requirements
- Let the AWS CDK manage roles and security groups
- Model all production stages in code
- Measure everything

Sharing resources between stacks

- You can refer to resources in a different stack as long as
  - they are defined in the same app and
  - They are in the same AWS account and Region
- The following pattern is generally used:
  - Store a reference to the construct as an attribute of the stack that produces the resource. (To get a reference to the current construct's stack, use Stack.of(this).)
  - Pass this reference to the constructor of the stack that consumes the resource as a parameter or a property. The consuming stack then passes it as a property to any construct that needs it.

Cross-stack resource sharing

- For example,
  - Stack1 defines an Amazon S3 bucket and stores a reference to the bucket construct as an attribute of the stack
  - Then the app defines a second stack, stack2, which accepts a bucket at instantiation. stack2 might, for example, uses the bucket for data storage

```
stack1 = StackThatProvidesABucket(app, "Stack1", env=prod)


# stack2 will take a property "bucket"
stack2 = StackThatExpectsABucket(app, "Stack2", bucket=stack1.bucket, env=prod)
```

CDK LAB: Cross-stack resource sharing

- Go back to the directory cdk-lab-web-server

- Create a network stack.
- Move VPC statement from the web server stack to the network stack
  - Add a property as a getter for other stacks to access the VPC

- Update the web server stack
  - Add one input parameter for VPC.
  - Make sure all statements reference the correct VPC

- Update app.py
  - Import the network stack
  - Instantiate network stack before the web server stack
  - Update the web server stack initialization to pass VPC from the network stack

Deploy and Test

- cdk diff
  - to see the difference
- cdk synth
  - to synthesize stacks
- cdk deploy --all
  - to deploy all stacks because we have more than one stack
- Examine all stacks and their resources